Patrones útiles de entradas

<table>
<tr><td colspan="2">1. Caso único (el más común)</td></tr>
<tr><td>Entrada:
n
datos...</td><td>Código:
int n;
cin >> n;</td></tr>
<tr><td>Regla clave (esto te sirve siempre)
Usa:
cin >> ...
scanf(...)</td><td>Formatos de scanf(...)
<table>
<tr><th>Tipo de dato</th><th>Variable</th><th>Formato scanf</th></tr>
<tr><td>int</td><td>int x;</td><td>%d</td></tr>
<tr><td>long long</td><td>long long x;</td><td>%lld</td></tr>
<tr><td>char</td><td>char c;</td><td>%c</td></tr>
<tr><td>string (char[])</td><td>char s[1000];</td><td>%s</td></tr>
<tr><td>float</td><td>float x;</td><td>%f</td></tr>
<tr><td>double</td><td>double x;</td><td>%lf</td></tr>
<tr><td>unsigned int</td><td>unsigned int x;</td><td>%u</td></tr>
<tr><td>unsigned long long</td><td>unsigned long long x;</td><td>%llu</td></tr>
</table></td></tr>
</table>

Ejemplos de algunos ejercicios de la plataforma DMOJ.UCLV	

https://dmoj.uclv.edu.cu/problem/evenodd
Entrada
 	
El número N
	int n;
	int main(){
	  cin>>n;
	}

https://dmoj.uclv.edu.cu/problem/ogroork
Entrada
La primera y única línea de entrada contiene dos números enteros a y b, la cantidad de hijos y de hijas que tiene el ogro Ork.
	int main (){
	  int a,b;
	  cin>>a>>b;
	}

https://dmoj.uclv.edu.cu/problem/bender **Entrada** La primera y única línea de la entrada contiene un número de cuatro cifras sin ceros al principio de su representación decimal.	**int** n; scanf("%d",&n);
https://dmoj.uclv.edu.cu/problem/notasork **Entrada** La **primera y única línea de entrada** está compuesta por un carácter, la nota de Ork.	**char** n; scanf("%c", &n);
https://dmoj.uclv.edu.cu/problem/bsum **Entrada** Leer un entero positivo $N \leq 1000000$	**int** N; cin>>N;

https://dmoj.uclv.edu.cu/problem/divide **Entrada** La entrada consiste en un número entero $1 \leq N \leq 10\ 000\ 000$	**int** N; cin>>N;
https://dmoj.uclv.edu.cu/problem/sum **Entrada** La entrada consiste en una sola línea con un entero N que no es mayor que 10^4 por su valor absoluto.	**int** N; cin>>N;
https://dmoj.uclv.edu.cu/problem/horasminutos **Entrada** La primera línea de la entrada contiene dos enteros $h(1 \leq h \leq 10^3)$ y $m(1 \leq m \leq 10^3)$representando la cantidad de horas y la cantidad de minutos.	**int** horas=0, minutos=0; scanf("%d %d",**&**horas, **&**minutos);
https://dmoj.uclv.edu.cu/problem/cad **Entrada** Una cadena de a lo más 100000 caracteres.	**const int** MAXN = 100001; **char** str[MAXN]; scanf("%s", str);

https://dmoj.uclv.edu.cu/problem/alexandioi
Entrada
La primera línea de la entrada contiene los enteros n, a, b $(1 \leq n \leq 10^5, 1 \leq a, b \leq n)$
La **segunda y última línea** contiene una cadena de caracteres de tamaño n la cual consiste solamente de los caracteres 0 y 1. Si el i-ésimo carácter es 0 el aeropuerto pertenece a la primera compañía, en otro caso el aeropuerto pertenece a la segunda compañía.
	int n, a, b;
	scanf("%d %d %d", &n, &a, &b);
	char str[n + 1];
	scanf("%s", str);

https://dmoj.uclv.edu.cu/problem/crne **Entrada** La primera línea de entrada contiene un número entero $N(1 \leq N \leq 100)$, el número de cortes que pueden hacer.	int n; scanf("%d", &n);
https://dmoj.uclv.edu.cu/problem/farnsworth **Entrada** Tres números enteros a, b, $c(1 \leq a, b, c \leq 100)$ que representan el tipo de los compuestos que tomó el profesor.	**int a,b,c;** **scanf("%i %i %i",&a,&b,&c);**
https://dmoj.uclv.edu.cu/problem/gusanillo **Entrada** En una línea, separados por un espacio, aparecen el tamaño de la rama R, la distancia D que recorre entre descansos y la distancia entre pares P consecutivos de hojas.	**int** rama, dist, pos_hojas; scanf("%i%i%i", **&**rama, **&**dist, **&**pos_hojas);

2. Múltiples casos con T

Número de casos conocido (T) ✓

Se usa cuando el enunciado dice algo como:
"La primera línea contiene un entero T (número de casos)"
"Se deben procesar T casos de prueba"
Es decir:
✓ Hay múltiples casos
✓ Sabes exactamente cuántos

Entrada:

T
caso1
caso2
...

¿Qué hace t--?

Va disminuyendo: 3 → 2 → 1 → 0
y el ciclo termina cuando llega a 0.

Código:

```
int t;
cin >> t;
while (t--) {
  // se ejecuta "n" veces (para procesar N caso)
}
```

También encaja perfectamente con for,
Muchos programadores incluso prefieren for.

¿Cuál es mejor?
while (t--): Más corto y muy usado en concursos.
For: **Más claro para principiantes.**

```
int t;
cin >> t;

for (int i = 0; i < t; i++) {

// Procesar un caso
}
```

Ejemplos de algunos ejercicios de la plataforma DMOJ.UCLV

https://dmoj.uclv.edu.cu/problem/aplusb

Entrada

La primera línea contendrá un entero N($1 \leq N \leq 100000$), es el número de problemas de suma que Tudor necesita hacer. Las próximas n líneas contendrán cada una dos enteros separados por espacio cuyo valor absoluto es menor que: $|x| < 1000000000$ representando los dos enteros que Tudor debe sumar.

¿Que tienes que tener en cuenta?
Clasificación del problema

- **Múltiples casos** con T
- Entrada tipo **bloque** simple
- Operación básica por caso (suma)

```
int N;
long long a, b;

cin >> N;
while (N--) {
   cin >> a >> b;
}
```

La primera línea de la entrada, contiene un número entero entre 1 y 100 000, que representa la cantidad de juegos de dominó que Neil recogerá hoy. Por cada juego de dominó que recogerá, la entrada contiene una línea con un número entero entre 4 y 100, que representa la cantidad máxima de puntos que podrá tener una sección de una ficha del juego en cuestión.	int main(){
	int c,l;
	cin>>c;
	while(c--)
	{
	cin>>l;
	}
	}

https://dmoj.uclv.edu.cu/problem/bellotas
Entrada
La entrada se dará en el siguiente formato:
• N
• $A_1, A_2, A_3, \dots, A_N$
	int main(){
	int n,a;
	cin>>n;
	while(n--)
	{
	cin>>a;
	}
	}

https://dmoj.uclv.edu.cu/problem/div6
Entrada
 	
La primera línea de la entrada tiene un entero T($1 \leq T \leq 25$), especificando el número de casos de prueba. Las siguientes T líneas tendrán un entero N con 1000 dígitos a lo sumo.
	int t;
	scanf("%d",&t);
	while(t--){
	char str[1001];
	scanf("%s",str);
	}

https://dmoj.uclv.edu.cu/problem/calculandoareas
 	
Una línea que contiene el número de casos T (1 <= T <= 100). Cada una de las siguientes líneas T contendrá el tamaño del lado (1 <= L <= 100) del cuadrado.
	int main(){
	int t;
	double L;
	scanf("%d",&t);
	while(t--){
	scanf("%lf",&L);
	}
	}

https://dmoj.uclv.edu.cu/problem/2019t1a
Entrada
 	
Un entero N (con $N \leq 100$)
Una línea con N enteros positivos $X_1, X_2, \dots, X_N$
Una línea con N enteros positivos $Y_1, Y_2, \dots, Y_N$
	int n;
	cin>>n;
	for(int i=0;i<n;i++){
	int x; cin>>x;
	}
	for(int i=0;i<n;i++){
	int y; cin>>y;
	}
	Nota: Sabes exactamente cuántas iteraciones hacer

https://dmoj.uclv.edu.cu/problem/22psn2c
Entrada
La primera línea de la entrada contendrá un **único número entero** positivo, $d(1 \leq d \leq 1000)$, que representa el número de días que Jenna hace prácticas de química. Luego, seguirán d **líneas**, cada una con **cuatro enteros**, ℓ_1, c_1, ℓ_2, y $c_2(0 \leq \ell_1 \leq 100;\ 0 \leq c_1 \leq 100;\ \ 0 \leq \ell_2 \leq 100;\ \ 0 \leq c_2 \leq 100)$, que representan la longitud y la complejidad del primer problema, y la longitud y la complejidad del segundo problema respectivamente.
	int d;
	scanf("%i", &d);
	int $c1$, $l1$, $c2$, $l2$;
	for (int i = 0; i < d; i++) {
	scanf("%i%i%i%i", &$c1$, &$l1$, &$c2$, &$l2$);
	}

https://dmoj.uclv.edu.cu/problem/frog
Entrada
La primera línea de la entrada contiene un número $T(1 \leq T \leq 100)$ de casos de prueba. Cada una de las siguientes T líneas representa un caso de prueba. Cada caso de prueba contiene un entero positivo con el número $N(1 \leq N \leq 10\ 000)$ de pasos correspondiente a un camino.
	int t, n;
	scanf("%d", &t);
	while (t--) {
	scanf("%d", &n);
	}

https://dmoj.uclv.edu.cu/problem/itoa
Entrada
La primera línea contiene el número entero N, el número de enteros, donde $0 \leq N \leq 1000$. Las N siguientes líneas contienen valores enteros X_n, donde $0 \leq X_n \leq 1\ 000\ 000$.
	int n, x;
	scanf("%d", &n);
	
	while(n--){
	scanf("%d", &x);
	}

https://dmoj.uclv.edu.cu/problem/haybales
Entrada
• **Línea 1**: El número de pilas, $N(1 \leq N \leq 10\ 000)$.
• **Líneas 2..1+N**: Cada línea contiene el número de fardos de heno en una sola pila (un entero en el rango $1 \ldots 10\ 000$).
	int t;
	cin>>t;
	while(t--){
	int a;
	cin>>a;
	}

🧠 3. Múltiples casos hasta EOF

No hay T (EOF) ✓

Se usa cuando: NO sabes cuántos casos vienen

El programa debe seguir leyendo: hasta que se termine la entrada

Eso se llama: EOF (End Of File), O **FIN DE FICHERO**

Es decir, tu no sabes cuantos hay por ejemplo:

Ejemplo: Dados varios números enteros, determine el valor máximo entre ellos.

Te das cuenta de que en ese enunciado, no dice cuantos números tienes que entrar, es la cantidad que le de la gana al juez entrar, porque **Tú NO sabes cuántos datos va a enviar el juez. Por tanto El programa debe seguir leyendo: hasta que ya no haya más entrada**

Cuando un enunciado:

- NO menciona T(es decir una cantidad exacta de líneas), es como no sabes cuando termina, porque NO da cantidad de casos
- **Ejemplo:**
  - "cada línea contiene…"
  - "para cada caso…"
  - "para cada número…"

📥 Entrada:

```
caso1
caso2
caso3
...
```

Codigo:

```
while (cin >> x) {
   // caso
}
```

```
int main() {
   int numero;
   while (scanf("%d", &numero) != EOF)   {
      //No hay T (EOF) ✓  while(cin >> …)
   }
   return 0;
}
```

***scanf → más rápido (C puro)

Ejemplo:

```
#include <iostream>
using namespace std;
int main() {
    int n, k;
    while (cin >> n >> k) {
        cout << "Suma: " << n + k << endl;
    }
    return 0;
}
```

**cin → más lento si no optimizas con

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

Ejercicios de Dmoj, donde se utiliza

https://dmoj.uclv.edu.cu/problem/coinchange

Entrada

El archivo de entrada contiene cualquier cantidad de líneas, cada una compuesta por un número representando la cantidad de dinero en centavos.

Solución

```
int m;
while(cin>>m){
  // Aquí procesas cada linea
}
```

https://dmoj.uclv.edu.cu/problem/fitawc

Entrada

Varias líneas, con **dos números** enteros **separados por un espacio** N **y** K, con $1 \leq K \leq N \leq 2000$.
Usted debe leer hasta alcanzar el final del archivo (EOF) y puede asumir que no habrá más de 10^4 líneas en la entrada.

Solución

```
int n,k;
while(cin>>n>>k){  // Leer mientras haya datos N y K
        // Aquí procesas cada linea
}
```

4. Leer hasta valor centinela (o Bandera)

Un **valor centinela** es un **valor especial que indica el final de la entrada**. Cuando se lee ese valor, el programa debe terminar la lectura.

Ejemplo: Que un programa termine cuando pongan 0 significa "termina"

Entrada:	Código:

5
8
3
0
	int x;
	while (cin >> x && **x != 0**) {
	// procesar
	}
	este indica que termina cuando escriba 0

Ejercicios de Dmoj, donde se utiliza

https://dmoj.uclv.edu.cu/problem/busfather

Entrada

La entrada contendrá varias líneas, **pero no más de 75**. En cada línea, **N** estará entre 1 y 10^9, inclusive.

Solución

```
int main() {
    long long N;
    int contador = 0;
    while (cin >> N && contador < 75) { // Leer hasta EOF pero limitando a 75 (por seguridad)
        // Aquí procesas cada N
         contador++;
    }
    return 0;
}
```

https://dmoj.uclv.edu.cu/problem/promediopuntos

Entrada

Cada fichero de entrada contiene a lo más **10 casos de prueba**. La entrada por cada caso de prueba consiste en dos enteros N $(2 \leq N \leq 60)$, el número de profesores, y K $(1 \leq K \leq 200)$ el máximo valor de puntuación que cualquier profesor puede dar. El mínimo valor que un profesor puede otorgar es siempre 0, cuando piensa que la presentación es un completo desastre :

Solución

```
int n,k;
int casosLeidos = 0;
while(cin>>n>>k && casosLeidos < 10) {  // Leer mientras haya datos y no se superen los 10 casos
        // Aquí procesas cada linea
    casosLeidos++;
}
```

🧠 5. Leer línea completa (strings con espacios). Leer múltiples líneas(Lee hasta EOF)

Si el problema trabaja con texto completo o frases

👉Importantísimo cuando hay espacios y necesitas todo el texto de cada línea, no lo necesitas separado sino junto

Cómo saberlo SIN adivinar

- Si el enunciado dice: "una cadena" → ❓ (puede ser con o sin espacios)
- Si dice: "una línea" → ✅ getline
- Si hay ejemplos como: hello world

Sintaxis básica

getline(cin, variable);

- cin → flujo de entrada
- variable → debe ser de tipo string

Entrada:
hola mundo
esto es prueba
	Código:
	string s;
	while (getline(cin, s)) {
	  // procesar línea completa
	}

cin >> n;
getline(cin, s); // ❌ problema
getline: Lee el salto de línea pendiente.
✔ Solución correcta:
cin >> n;
cin.ignore();
getline(cin, s);
	getline(cin, s)
	Cuando:
	• El input puede tener espacios
	• O es una **línea completa de texto**
	• Ejemplo: cow on the way to zoo

#include <iostream>
#include <string>
using namespace std;
int main() {
  string s;
  getline(cin, s);
  cout << s << endl;
  return 0;
}
input: hola mundo cruel
Se guarda completo en s:
"hola mundo cruel"
	Sintaxis con delimitador (más avanzada)
	getline(cin, s, delimitador);
	
	string s;
	getline(cin, s, ',');
	
	input: hola,mundo,adios
	s = "hola"

Si usas cin >> antes, Solución correcta es:
int n;
cin >> n;
cin.ignore(); // limpia el buffer
getline(cin, s);
	Leer múltiples líneas(Lee hasta EOF)
	string s;
	while (getline(cin, s)) {
	  cout << s << endl;
	}

Sino pones cin.ignore(); **getline** lee el salto de línea pendiente y queda vacío. Por eso es que se usa para **Limpiar el buffer**	

Ejercicios de Dmoj, donde se utiliza	

https://dmoj.uclv.edu.cu/problem/cowoytz
Entrada
• **Línea 1:** Tres enteros separados por espacios: N, S y E.
• **Líneas 2..E+1:** La línea i+1 describe la expresión i como se ha especificado anteriormente.
	string line;
	int N, S, E;
	cin >> N >> S >> E;
	for (**int** i = 0; i < E; i++) {
	getline(cin, line);
	}

https://dmoj.uclv.edu.cu/problem/chkr
Entrada
• Línea 1: Un solo entero: N.
• Líneas 2..N+1: La línea i+1 contiene N caracteres ('-', '+', 'K', 'o') que representan la fila i de un tablero apropiado. La línea 2 siempre comienza con '-'.
	int N; string fila;
	cin >> N;
	cin.ignore();
	for (**int** i = 0; i < N; i++) {
	getline(cin, fila);
	}

https://dmoj.uclv.edu.cu/problem/elpoema
Entrada
 	
La primera línea tendrá un número entero N ($1 < N < 1000$) que representa el número de líneas que tendrá el poema.
Cada línea del poema tendrá un máximo de 100 caracteres que contendrá: mayúsculas ($A - Z$), minúsculas ($a - z$), espacios ( ), puntos (.) y comas (,).
	string linea;
	int N;
	cin >> N;
	cin.**ignore**();
	while(N--){
	getline(cin, linea);
	// Aquí procesas cada línea
	}

https://dmoj.uclv.edu.cu/problem/foxy **Entrada** La **primera línea** de entrada contiene el **número** de **casos** de prueba T. Las descripciones de los casos de prueba a continuación: La primera línea de cada caso de prueba contiene la grabación: palabras sobre el alfabeto inglés en minúsculas, separadas por espacios. Cada uno contiene a lo sumo 100 letras y no hay más de 100 palabras. Las siguientes	1ra linea la grabación: palabras sobre el alfabeto inglés en minúsculas, separadas por espacios. Cada uno contiene a lo sumo 100 letras y no hay más de 100 palabras. Las siguientes líneas son información de otros animales, en el formato <animal> goes <sonido>. No hay más de 100 animales, sus

líneas son su información recopilada previamente sobre otros animales, en el formato <animal> goes <sonido>. No hay más de 100 animales, sus nombres no tienen más de 100 letras cada uno y son nombres reales de animales en inglés. No hay ningún zorro entre estas líneas. La última línea del caso de prueba es exactamente la pregunta que se supone que debes responder: ¿qué dijo el zorro?	nombres no tienen más de 100 letras cada uno y son nombres reales de animales en inglés. No hay ningún zorro entre estas líneas. La última línea del caso de prueba es exactamente la pregunta que se supone que debes responder: ¿qué dijo el zorro?

```
    int T;
    cin >> T;
    cin.ignore();
    while (T--) {
        string grabacion;
        getline(cin, grabacion); // Leer la grabación (1ra linea del caso)

        string a, b, c;

        // Leer información de otros animales hasta encontrar "what does the fox say?"
        while (cin >> a >> b >> c) {
            if (a == "what" && b == "does" && c == "the") {
                string d, e;
                cin >> d >> e; // Leemos "fox say?" (última parte)
                break;
            }
            // Aquí guardarías el sonido (c) para cada animal (a)
        }
    }

    return 0;
}
```

https://dmoj.uclv.edu.cu/problem/explora23p1bb **Entrada** La entrada consiste de un conjunto de dos o más puntos únicos, donde las coordenadas cartesianas de cada punto se especifican en una línea separada usando dos enteros X y Y. $0 \leq \mid X \mid, \mid Y \mid < 1{,}000{,}000$. Ningún caso tiene más de 1000 puntos. La entrada termina con una línea con dos o más signos menos **"--"**.	

```
int main() {
    string linea;
    int X, Y;

    while (getline(cin, linea)) {
        // Si la línea es "--" o comienza con "--", terminamos
        if (linea == "--" || linea.find("--") == 0) {
            break;
        }

        // Intentar leer los dos enteros de la línea
        if (sscanf(linea.c_str(), "%d %d", &X, &Y) == 2) {
            // Procesar el punto (X, Y)
        }
    }

    return 0;
}
```

https://dmoj.uclv.edu.cu/problem/2023occcop10c

Entrada

Línea 1: se darán 4 enteros n (columnas), m (filas), x e y.

Línea 2..2+n: n líneas de m longitud con un signo de punto (.) para representar el espacio vacío o un signo de número (#) para representar la paca de tabaco en la pirámide.

```
    int n, m, x, y;
    string linea;

    getline(cin, linea); // Línea 1
    stringstream ss(linea);
    ss >> n >> m >> x >> y;

   int i = 0;
    while (i < n) {
        getline(cin, linea); // Línea 2..2+n

   // Saltar líneas vacías
   if (linea.empty()) continue;

// Validar que la línea tenga exactamente m caracteres
        while (linea.length() != m) {
            getline(cin, linea);
        }
        i++;
    }

    return 0;
}
```

<table>
<tr><td colspan="2">😵 6. Mezcla cin + getline (TRAMPA clásica). Leer múltiples líneas. Número de casos conocido (T)</td></tr>
<tr><td>Entrada:
3
hola mundo
adios mundo
test</td><td>Código:
int n;
cin >> n;
cin.ignore(); //
string s;
for (int i = 0; i < n; i++) {
  getline(cin, s);
}</td></tr>
<tr><td colspan="2">Ya de este no puse ningún ejemplo, porque aquí es casi lo mismo de arriba, solo que debes RECORDAR QUE SIEMPRE QUE PONES UN cin, debes poner detrás un ignore.. Porque Si no haces cin.ignore() → falla…. Y se debe a que después de cada cin deja un \n al final</td></tr>
</table>

7. Leer matriz / grid

Entrada:

n m

grid...

Código:

```
int n, m;
cin >> n >> m;
vector<string> grid(n);
for (int i = 0; i < n; i++) {
   cin >> grid[i];
}
```

https://dmoj.uclv.edu.cu/problem/spacedout

Entrada

La primera línea contiene N. Cada una de las siguientes N líneas contiene N enteros. El entero j ésimo de la fila i ésima desde arriba es el valor de a_{ij}

```
int grid[100][100];
int n;
   cin >> n;
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         cin >> grid[i][j];
      }
   }
```

https://dmoj.uclv.edu.cu/problem/loteria

Entrada

La primera línea de la entrada estándar contiene dos enteros separados por espacios n y l ($1 \leq l \leq n \leq 10\,000$), el **número de días** y **duración** de los **intervalos** analizados.

La segunda línea contiene n enteros separados por espacios $a_1, a_2, \ldots, a_n$ ($1 \leq a_i \leq 10^9$), donde a_i es el número que se dibujó en el i-ésimo día.

La tercera línea contiene un entero q ($1 \leq q \leq 100$), el número de consultas. Cada una de las siguientes q líneas contiene un entero k_j ($0 \leq k_j \leq l$), el parámetro de similitud para la j-ésima consulta.

```
int n, l, q;
vector<int> tab;
cin >> n >> l;
 tab.resize(n); //  Redimensiona con n
 for (int i = 0; i < n; i ++) cin >> tab[i];  // Línea 2:
    cin >> q; //Línea 3:

 for (int i = 0; i < q; i ++) { // Siguientes q líneas: cada una con kj
    int x;
    cin >> x;
 }
```

Entrada

- La primera línea contiene dos enteros n y k: el tamaño de la cuadrícula y el número de letras. Las letras son las primeras k letras mayúsculas.
- A continuación, hay n líneas que describen la cuadrícula. Cada línea contiene n letras.

```
   int n,k;
   cin >> n >> k;
   vector<string> grid(n);
   for(int i=0;i<n;i++)
      cin >> grid[i];
    }
```